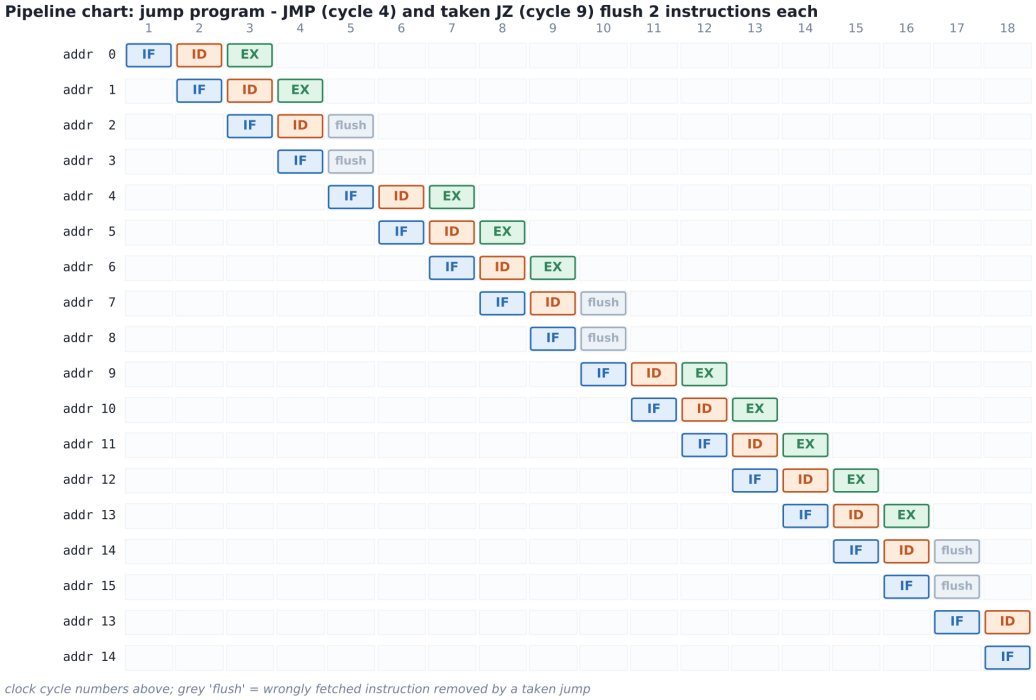
Code walkthrough

tb_cpu.v

Testbench: Complete CPU

What it does	Runs 4 programs plus a reset test (44 checks)
Where it works	CPU-level test
Size	277 lines, explained in 16 small parts
Modules in this file	tb_cpu
Calls (uses)	cpu_top.v (module cpu_top), which uses all other design files; programs program.mem, prog_arith.mem, prog_logic.mem, prog_jump.mem
Called by	nobody - this is a testbench (top of simulation)

This is the main testbench. It runs programs on the complete pipelined CPU and checks the result after important instructions. It also prints what is in each pipeline stage, so you can watch the pipeline work.



Pipeline chart produced from tb_cpu Test 4 (jumps and flushing).

Map of the file

Part	Lines	What this part is
1	1-19	Header
2	20-30	Signals and clock
3	32-41	The CPU under test
4	43-47	Cycle counter
5	49-72	Trace printing
6	74-85	Task: reset_cpu
7	87-101	Task: load_program
8	103-116	Task: finish
9	118-132	Task: check
10	134-148	Task: check_bit
11	150-173	Test 1: demonstration program
12	175-196	Test 2: arithmetic and memory
13	198-217	Test 3: logic
14	219-242	Test 4: PC and jumps
15	244-266	Test 5: reset and unknown values
16	268-277	Summary

How to read this document: each part below shows a few lines of the real file, with their line numbers, followed by a plain-language explanation. Read the parts in order to understand the whole file without scrolling through it in one go.

Part 1 · Header (lines 1-19)

tb_cpu.v lines 1-19

```
 1  `timescale 1ns / 1ps
 2  //
 3  // CONNECTIONS
 4  //   Calls       : cpu_top.v (module cpu_top), which uses all other design files;
 5  //                 programs program.mem, prog_arith.mem, prog_logic.mem, prog_jump.mem
 6  //   Called by   : nobody - this is a testbench (top of simulation)
 7  // ========================================================================
 8  // FILE: tb_cpu.v
 9  // CPU-LEVEL TEST: runs programs on the complete pipelined CPU and checks
10  // the result after each important instruction (project plan, Section 8).
11  //
12  //   TEST 1  program.mem    : demonstration program (plan Appendix A)
13  //                            + a cycle-by-cycle view of the pipeline
14  //   TEST 2  prog_arith.mem : ADD, SUB, Carry, STORE, LOAD, Zero
15  //   TEST 3  prog_logic.mem : AND, OR, XOR, MOV, NOT, LDI, Zero
16  //   TEST 4  prog_jump.mem  : PC increment, JMP to 0100, JZ taken/not taken,
17  //                            flushing of wrongly fetched instructions
18  //   TEST 5  reset          : reset values and no unknown (X) values
19  // ========================================================================
```

- Lists the five tests and which program file each one uses.
- The **CONNECTIONS** lines at the end of the header list which other files this file uses (Calls) and which files use it (Called by).

Part 2 · Signals and clock (lines 20-30)

tb_cpu.v lines 20-30

```
20  module tb_cpu;
21
22      reg clk = 0;
23      reg rst = 1;
24      integer errors = 0;
25      integer checks = 0;
26      integer cycle  = 0;            // clock cycles since reset was released
27      integer i;
28      reg     show_pipeline = 0;     // 1 = print every clock cycle
29
30      always #5 clk = ~clk;          // 10 ns clock period
```

- Counters for errors and checks, a `cycle` counter, and a switch `show_pipeline` that turns on the detailed per-cycle print-out.
- A 10 ns clock.

Part 3 · The CPU under test (lines 32-41)

tb_cpu.v lines 32-41

```
32      wire [3:0]  pc_out, alu_out, wb_data_out;
33      wire [11:0] instr_out;
34      wire        zero_out, carry_out;
35
36      // CALLS: module cpu_top from file cpu_top.v
37      cpu_top #(.PROGRAM_FILE("program.mem")) CPU (
38          .clk(clk), .rst(rst),
39          .pc_out(pc_out), .instr_out(instr_out), .alu_out(alu_out),
40          .wb_data_out(wb_data_out), .zero_out(zero_out), .carry_out(carry_out)
41      );
```

One complete CPU, starting with program.mem. Its debug outputs are connected to wires so they can be checked.

→ This part **calls module** `cpu_top` from file `cpu_top.v`. How that module works is explained in `Explained_cpu_top.pdf`.

Part 4 · Cycle counter (lines 43-47)

tb_cpu.v lines 43-47

```
43      // count clock cycles
44      always @(posedge clk) begin
45          if (rst) cycle <= 0;
46          else     cycle <= cycle + 1;
47      end
```

Counts rising clock edges after reset. Used to show how many cycles a program takes.

Part 5 · Trace printing (lines 49-72)

tb_cpu.v lines 49-72

```
49      // ------------------------------------------------------------------
50      // Trace. Printed in the middle of each clock cycle.
51      //   show_pipeline = 1 : what each stage holds, every cycle
52      //   otherwise         : one line when an instruction is in EXECUTE
53      // CPU.RF.r1 means: inside CPU (cpu_top.v), inside RF (register_file.v),
54      // the signal r1. CPU.id_instr / CPU.id_valid come from
55      // instruction_register.v (instance IR); CPU.ex_instr / CPU.ex_valid
56      // come from pipeline_register.v (instance ID_EX).
57      // ------------------------------------------------------------------
58      always @(negedge clk) begin
59          if (!rst && show_pipeline) begin
60              $write("   cycle %2d | IF: addr %2d | ID: ", cycle + 1, CPU.pc);
61              if (CPU.id_valid) $write("%h (addr %2d)", CPU.id_instr, CPU.id_pc);
62              else              $write("bubble        ");
63              $write(" | EX: ");
64              if (CPU.ex_valid) $write("%h (addr %2d)", CPU.ex_instr, CPU.ex_pc);
65              else              $write("bubble        ");
66              $display(" | R1=%b R2=%b R3=%b", CPU.RF.r1, CPU.RF.r2, CPU.RF.r3);
67          end
68          else if (!rst && CPU.ex_valid)
69              $display("   EX addr %2d: %h | R0=%b R1=%b R2=%b R3=%b | Z=%b C=%b",
70                       CPU.ex_pc, CPU.ex_instr, CPU.RF.r0, CPU.RF.r1, CPU.RF.r2, CPU.RF.r3,
71                       zero_out, carry_out);
72      end
```

- Runs at every falling clock edge (the middle of the cycle, when values are stable).
- If show_pipeline is on: prints the address in IF, the instruction in ID and the instruction in EX (or "bubble" if the slot is empty), plus R1-R3.
- Otherwise: prints one line whenever a real instruction is in EX.
- `CPU.id_instr` is a hierarchical name: the signal id_instr inside the CPU instance. This only works in simulation.

Part 6 · Task: reset_cpu (lines 74-85)

tb_cpu.v lines 74-85

```
74      // ====================================================================
75      // Helper tasks
76      // ====================================================================
77
78      // reset the CPU for 2 clocks
79      task reset_cpu;
80          begin
81              rst = 1;
82              @(posedge clk); @(posedge clk); #1;
83              rst = 0;
84          end
85      endtask
```

Holds reset for two clock edges and then releases it.

Part 7 · Task: load_program (lines 87-101)

tb_cpu.v lines 87-101

```
 87      // NOTE: CPU.IMEM.mem is the array 'mem' inside instruction_memory.v
 88      //       (instance IMEM in cpu_top.v); CPU.DMEM.mem is the array 'mem'
 89      //       inside data_memory.v (instance DMEM in cpu_top.v).
 90      // load a program file and clear the data memory
 91      task load_program;
 92          input [8*20:1] file_name;
 93          begin
 94              rst = 1;
 95              for (i = 0; i < 16; i = i + 1) begin
 96                  CPU.IMEM.mem[i] = 12'hC00;
 97                  CPU.DMEM.mem[i] = 4'b0000;
 98              end
 99              $readmemh(file_name, CPU.IMEM.mem);
100          end
101      endtask
```

Fills the instruction memory with NOP, clears the data memory, then loads a new .mem file. This lets one testbench run several programs.

Part 8 · Task: finish (lines 103-116)

tb_cpu.v lines 103-116

```
103      // NOTE: CPU.ex_valid and CPU.ex_pc are outputs of pipeline_register.v
104      //       (instance ID_EX in cpu_top.v).
105      // wait until the instruction at address 'addr' has finished
106      // (it is in EXECUTE, and the next clock edge writes its result)
107      task finish;
108          input [3:0] addr;
109          begin
110              @(negedge clk);
111              while (!(CPU.ex_valid && CPU.ex_pc == addr))
112                  @(negedge clk);
113              @(posedge clk);
114              #1;
115          end
116      endtask
```

- Waits until the instruction at a given address is in EX, then waits for the clock edge that writes its result.
- Because jumps change the timing, waiting for an instruction is more reliable than counting cycles.

Part 9 · Task: check (lines 118-132)

tb_cpu.v lines 118-132

```
118      // compare a 4-bit value
119      task check;
120          input [8*44:1] name;
121          input [3:0]    actual;
122          input [3:0]    expected;
123          begin
124              checks = checks + 1;
125              if (actual === expected)
126                  $display("PASS  %0s = %b", name, actual);
127              else begin
128                  $display("FAIL  %0s = %b (expected %b)", name, actual, expected);
129                  errors = errors + 1;
130              end
131          end
132      endtask
```

Compares a 4-bit value and prints PASS or FAIL. === also catches unknown (X) values.

Part 10 · Task: check_bit (lines 134-148)

tb_cpu.v lines 134-148

```
134      // compare a 1-bit value
135      task check_bit;
136          input [8*44:1] name;
137          input          actual;
138          input          expected;
139          begin
140              checks = checks + 1;
141              if (actual === expected)
142                  $display("PASS  %0s = %b", name, actual);
143              else begin
144                  $display("FAIL  %0s = %b (expected %b)", name, actual, expected);
145                  errors = errors + 1;
146              end
147          end
148      endtask
```

The same for single-bit values such as flags.

Part 11 · Test 1: demonstration program (lines 150-173)

tb_cpu.v lines 150-173

```
150     // ================================================================
151     // Tests
152     // ================================================================
153     initial begin
154
155         // The tasks used below (reset_cpu, load_program, finish, check,
156         // check_bit) are defined above in this same file.
157
158         // ---------------- TEST 1: demonstration program ----------------
159         $display("\n===== TEST 1: demonstration program (program.mem) =====");
160         show_pipeline = 1;
161         reset_cpu;
162         finish(2);
163         check("ADD R1,R2  -> R1 (5+3)", CPU.RF.r1, 4'b1000);
164         finish(4);
165         check("LDI R3,1010 -> R3", CPU.RF.r3, 4'b1010);
166         check("XOR R1,R3  -> R1", CPU.RF.r1, 4'b0010);
167         check("R2 still 0011", CPU.RF.r2, 4'b0011);
168         $display("      5 instructions finished after %0d clock cycles", cycle);
169         finish(5);
170         check("JMP 0101: PC (fetch address) = 5", pc_out, 4'd5);
171         finish(5);
172         show_pipeline = 0;
173         check("JMP runs again: program loops at 5", pc_out, 4'd5);
```

- Runs the plan's Appendix A program with the per-cycle pipeline trace turned on.
- Checks ADD (R1 = 1000), LDI (R3 = 1010) and XOR (R1 = 0010), prints the cycle count, and checks that JMP keeps the program looping at address 5.

Part 12 · Test 2: arithmetic and memory (lines 175-196)

tb_cpu.v lines 175-196

```
175         // ---------------- TEST 2: arithmetic and memory ----------------
176         $display("\n===== TEST 2: arithmetic and memory (prog_arith.mem) =====");
177         load_program("prog_arith.mem");
178         reset_cpu;
179         finish(2);
180         check    ("ADD R1,R2 (0101+0011) -> R1", CPU.RF.r1, 4'b1000);
181         check_bit("ADD: Carry", carry_out, 1'b0);
182         check_bit("ADD: Zero", zero_out, 1'b0);
183         finish(3);
184         check    ("STORE R1,0 -> data_memory[0]", CPU.DMEM.mem[0], 4'b1000);
185         finish(5);
186         check    ("SUB R1,R2 (0101-0011) -> R1", CPU.RF.r1, 4'b0010);
187         check_bit("SUB: Zero", zero_out, 1'b0);
188         check_bit("SUB: Carry (1 = no borrow)", carry_out, 1'b1);
189         finish(6);
190         check    ("STORE R1,1 -> data_memory[1]", CPU.DMEM.mem[1], 4'b0010);
191         finish(7);
192         check    ("LOAD R0,0 -> R0", CPU.RF.r0, 4'b1000);
193         finish(8);
194         check    ("SUB R2,R2 -> R2", CPU.RF.r2, 4'b0000);
195         check_bit("Zero flag after 0000 result", zero_out, 1'b1);
196         $display("      9 instructions finished after %0d clock cycles (pipeline: 9 + 2)", cycle);
```

Loads prog_arith.mem. Checks ADD with its flags, STORE, SUB with Carry = 1 (no borrow), a second STORE, LOAD, and a SUB that gives zero and sets the Zero flag.

Part 13 · Test 3: logic (lines 198-217)

tb_cpu.v lines 198-217

```
198         // ---------------- TEST 3: logic, MOV, NOT, LDI ----------------
199         $display("\n===== TEST 3: logic operations (prog_logic.mem) =====");
200         load_program("prog_logic.mem");
201         reset_cpu;
202         finish(3);
203         check    ("AND 1100,1010 -> R3", CPU.RF.r3, 4'b1000);
204         finish(5);
205         check    ("OR  1100,1010 -> R3", CPU.RF.r3, 4'b1110);
206         finish(6);
207         check    ("XOR 1100,1010 -> R1", CPU.RF.r1, 4'b0110);
208         finish(7);
209         check    ("MOV R2,R1 (R1=0110) -> R2", CPU.RF.r2, 4'b0110);
210         finish(9);
211         check    ("NOT 0101 -> R0", CPU.RF.r0, 4'b1010);
212         finish(10);
213         check    ("LDI R3,1010 -> R3", CPU.RF.r3, 4'b1010);
214         finish(11);
215         check    ("XOR R3,R3 -> R3", CPU.RF.r3, 4'b0000);
216         check_bit("Zero flag after 0000 result", zero_out, 1'b1);
217         $display("      12 instructions finished after %0d clock cycles (pipeline: 12 + 2)", cycle);
```

Loads prog_logic.mem. Checks AND, OR, XOR, MOV, NOT, LDI and a zero result from XOR.

Part 14 · Test 4: PC and jumps (lines 219-242)

tb_cpu.v lines 219-242

```
219         // ---------------- TEST 4: PC increment and jumps ----------------
220         $display("\n===== TEST 4: PC and jumps (prog_jump.mem) =====");
221         load_program("prog_jump.mem");
222         rst = 1;
223         @(posedge clk); @(posedge clk); #1;
224         check    ("PC after reset", pc_out, 4'd0);
225         rst = 0;
226         @(posedge clk); #1;
227         check    ("PC increment (0 -> 1)", pc_out, 4'd1);
228         finish(1);
229         check    ("JMP 4: PC becomes 0100", pc_out, 4'b0100);
230         check_bit("JMP flushed the ID/EX register", CPU.ex_valid, 1'b0);
231         finish(5);
232         check_bit("SUB R1,R1 sets Zero", zero_out, 1'b1);
233         finish(6);
234         check    ("JZ 9 taken: PC = 9", pc_out, 4'd9);
235         finish(10);
236         check_bit("ADD R3,R3 clears Zero", zero_out, 1'b0);
237         finish(11);
238         check    ("JZ 14 not taken: next in EX is addr 12", CPU.ex_pc, 4'd12);
239         finish(13);
240         check    ("R0 = 1 (JMP skipped addresses 2, 3)", CPU.RF.r0, 4'd1);
241         check    ("R2 = 6 (JZ skipped 7, 8; 14 never ran)", CPU.RF.r2, 4'd6);
242         check    ("Program stops at PC = 13", pc_out, 4'd13);
```

- Checks PC = 0 after reset and the PC increment.
- JMP 4: the PC becomes 0100 and the ID/EX register is flushed.
- JZ 9 taken (Zero = 1) and JZ 14 not taken (Zero = 0).
- R0 = 1 and R2 = 6 prove that every skipped instruction really never ran.

Part 15 · Test 5: reset and unknown values (lines 244-266)

tb_cpu.v lines 244-266

```
244         // ---------------- TEST 5: reset and unknown values ----------------
245         $display("\n===== TEST 5: reset =====");
246         rst = 1;
247         @(posedge clk); #1;
248         check    ("PC after reset", pc_out, 4'd0);
249         check    ("R0 after reset", CPU.RF.r0, 4'd0);
250         check    ("R2 after reset", CPU.RF.r2, 4'd0);
251         check_bit("Zero after reset", zero_out, 1'b0);
252         check_bit("Carry after reset", carry_out, 1'b0);
253         check_bit("IF/ID empty after reset", CPU.id_valid, 1'b0);
254         check_bit("ID/EX empty after reset", CPU.ex_valid, 1'b0);
255         rst = 0;
256         repeat (6) @(posedge clk);
257         #1;
258         // ^{...} XORs all bits together: the answer is X if ANY bit is X
259         checks = checks + 1;
260         if ((^{pc_out, instr_out, alu_out, wb_data_out, zero_out, carry_out,
261                CPU.RF.r0, CPU.RF.r1, CPU.RF.r2, CPU.RF.r3}) === 1'bx) begin
262             $display("FAIL  unknown (X) value found on key outputs");
263             errors = errors + 1;
264         end
265         else
266             $display("PASS  no unknown (X) values on key outputs or registers");
```

- Asserts reset and checks PC, registers, flags and both pipeline registers.
- Runs a few cycles, then XORs all key bits together with `^{...}`. The answer is X only if some bit is X, so this one test proves there are no unknown values.

Part 16 · Summary (lines 268-277)

tb_cpu.v lines 268-277

```
268         // ---------------- summary ----------------
269         $display("\n================================================");
270         if (errors == 0)
271             $display("tb_cpu PASSED: %0d of %0d checks OK", checks, checks);
272         else
273             $display("tb_cpu FAILED: %0d of %0d checks failed", errors, checks);
274         $display("================================================\n");
275         $finish;
276     end
277 endmodule
```

Prints PASSED or FAILED with the number of checks and stops the simulation.

> **In short:** 44 checks across 4 programs and a reset test, all passing.